Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

Department of Information Technology and Electrical Engineering

Machine Learning on Microcontrollers

227-0155-00L

Exercise 9

CIFAR-10 Image Classification on IMX500

ETH Center for Project-Based Learning

Wednesday 26th November, 2025

1 Introduction

In this lab we are going to discover the IMX500 chip from Sony Semiconductor Devices. The IMX500 combines a CMOS image sensor and a digital signal processor (DSP) in one chip, where the DSP is optimized for convolutional neural networks (CNN). Our goal is to do image classification on the CIFAR-10 [1] dataset and visualize the classification result as well as the camera image live. The training is done on Google Colab. We will further use the Raspberry Pi AI camera[2], which includes the IMX500 and allows for easy connection with a Raspberry Pi.

The goal of this lab is to walk you through the entire process. As such, we will train the model, evaluate it, convert it to run on the Raspberry Pi, create an application for the Raspberry Pi and last but definitely not least, characterize the key metrics of the model.

2 Notation

Student Task 1: Parts of the exercise that require you to complete a task will be explained in a shaded box like this.

Note: You find notes and remarks in boxes like this one.

3 Preparation

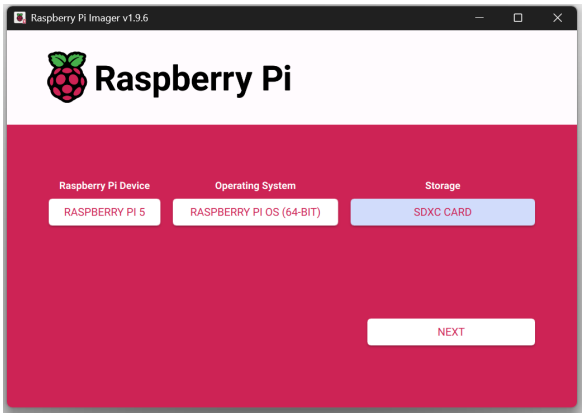
First, we need to setup both our training environment as well as the Raspberry Pi with the camera. Due to the computational requirements, we will use Google Colab for training. The Raspberry Pi is solely used to convert our model into the correct file format and run our application.

Note: Note that in the following tutorial, everything that you have to do, is specifically mentioned. That means you don't need to run a cell in the Jupyter notebook if it's not specifically mentioned that you have to run it.

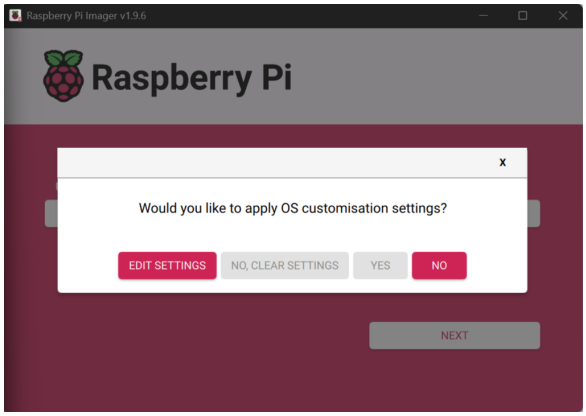
3.1 Raspberry Pi and Raspberry Pi AI Camera

To interface the IMX500, we use a Raspberry Pi. The following guide goes through the installation of the Raspberry Pi. Note that with this guide you don't need an external screen, as the Raspberry Pi is booted "headless" from the beginning on.

[1] https://www.cs.toronto.edu/ kriz/cifar.html
[2] https://www.raspberrypi.com/documentation/accessories/ai-camera.html



(a) Select the Raspberry Pi 5, Raspberry Pi OS 64-Bit version and the connected SD-card.



(b) Click on `EDIT SETTINGS` to configure detailed settings.

Figure 1: Raspberry Pi Imager general settings

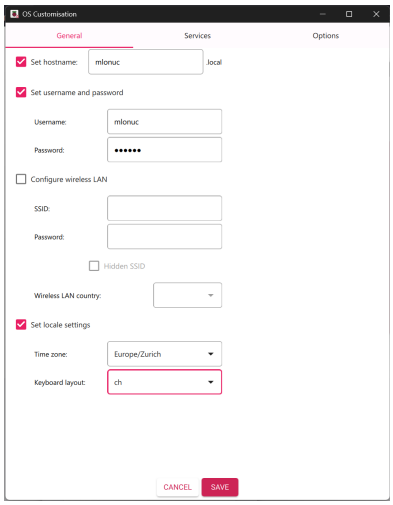
3.1.1 Imaging the Operating System onto the SD-Card

In the following we will prepare the SD-card for the Raspberry Pi.

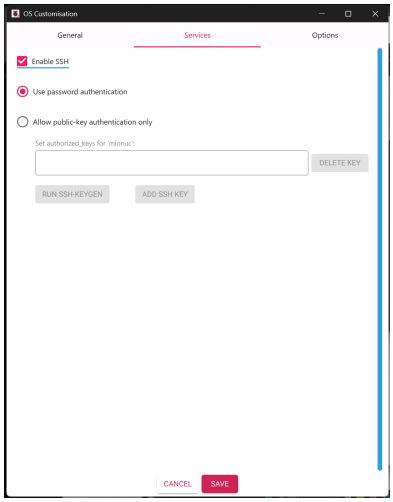
Student Task 2: The goal of this task is to prepare the SD-card for the Raspberry Pi.

- Download the Raspberry Pi Imager from [a] and install it as described in the link.
- Plug in a SD-card to write the image to. Note that all data on this SD-card will be erased permanently!
- Apply the settings as seen in Figure 1a.
- Click `NEXT` and on the next view as in Figure 1b click on `EDIT SETTINGS`.
- On the next screen as in Figure 2a, set the hostname, username, password and locale settings accordingly. In this example we use `mlonuc` as username and password. If you prefer you can use your own unique and secure password. It is recommended to always use secure passwords, also in a test environment.
- Change to the `Service` tab as in Figure 2b and enable SSH with password authentication. Note that when connecting the Raspberry Pi to a network, `public-key authentication only` would be more secure than `password authentication`, but this is out of scope for this guide.
- Go to the next tab, as in Figure 2c and disable telemetry.
- Save the settings.
- You can click on `NEXT` and when you are sure that the content of the SD-card can be erased, you click on `YES` to write the image to the SD-card, as seen in Figure 3a.
- It will now show the progress and once it has successfully finished, you will see Figure 3b. Now you can close the Raspberry Pi Imager and remove the SD-card. As we set the Imager to automatically eject the SD-card you can directly remove it from the computer without further actions.

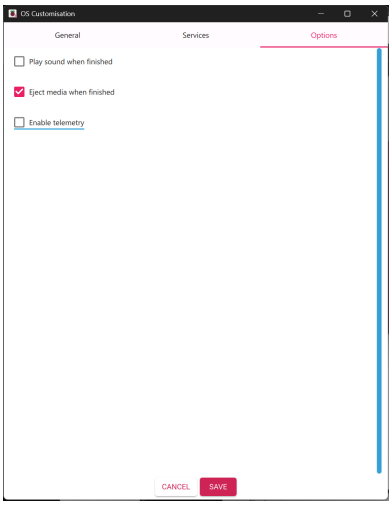
[a] https://www.raspberrypi.com/software/



(a) Set the hostname, username, password and locale settings.

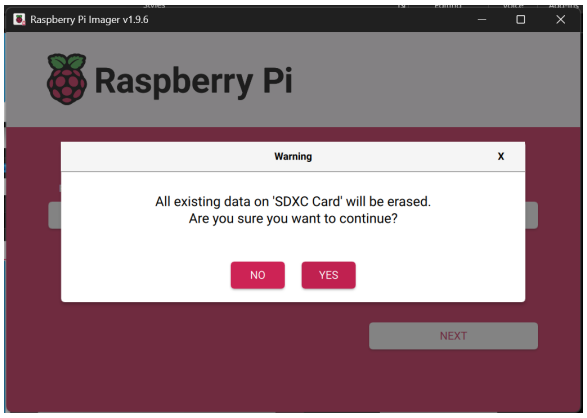


(b) Enable SSH and the use of password authentication. Note that SSH password authentication is seen as vulnerable in productive setups, as such public-key authentication would be preferred for this.

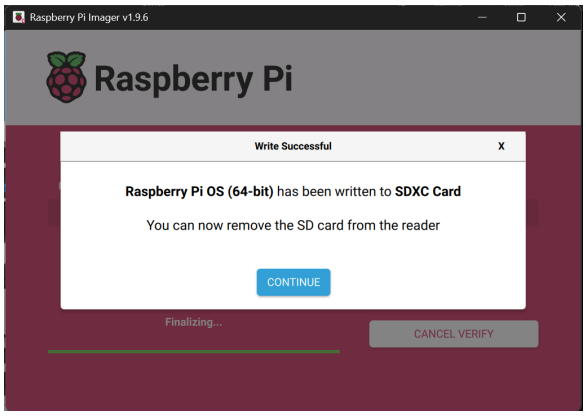


(c) Disable telemetry.

Figure 2: Raspberry Pi Imager detailed settings.



(a) Click on `YES` to write to the SD-card. Attention: All data will be erased from the SD-card.



(b) Wait till the write to the SC-card has successfully finished, click `CONTINUE` and close the Imager. Due to our previous setting, the SD-card will be automatically ejected and you can remove it from your computer.

Figure 3: Raspberry Pi Imager writing to the SD-card

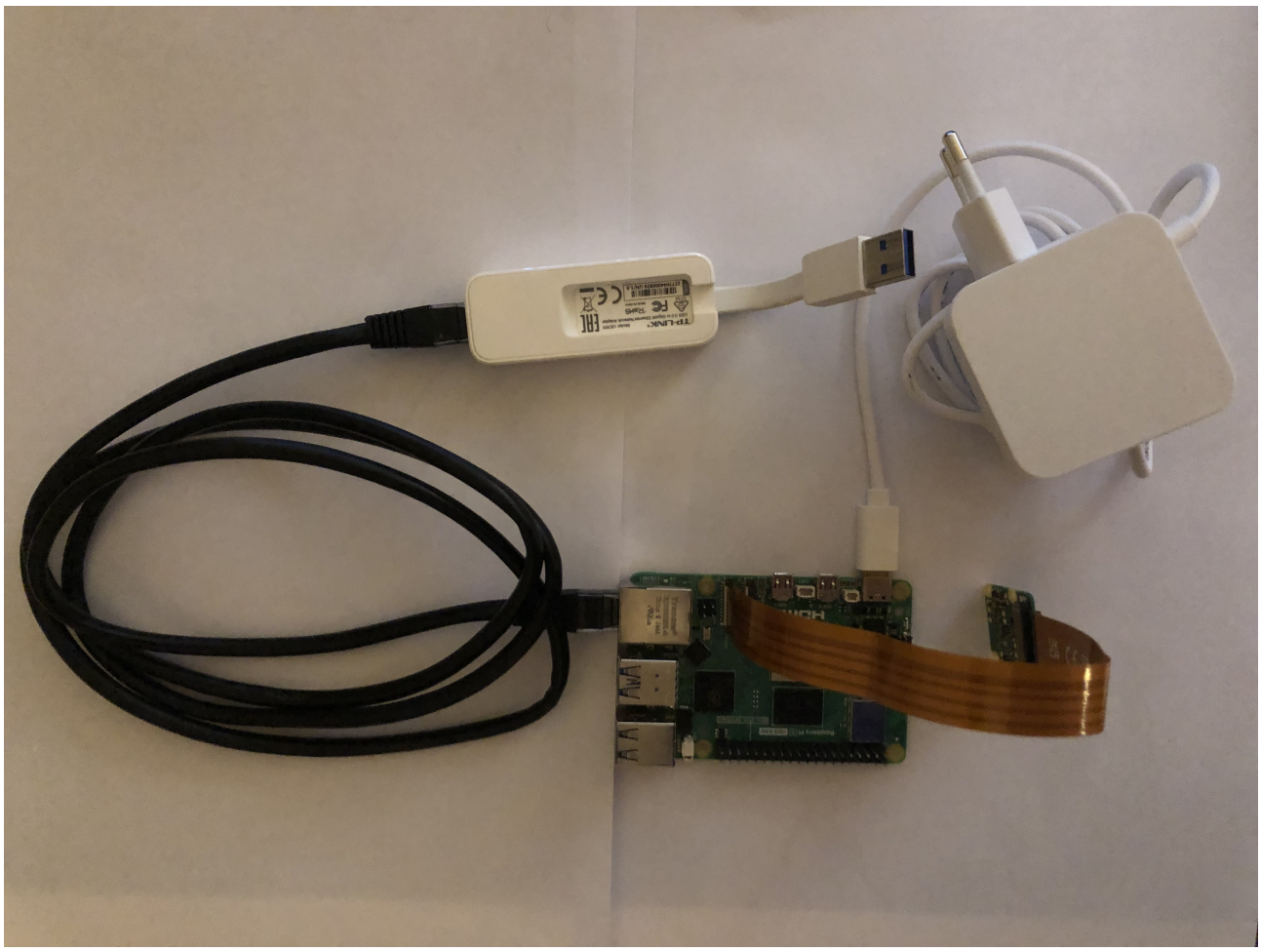


Figure 4: The connected Raspberry Pi.

3.1.2 Hardware Setup of Raspberry Pi

In the following task, you will connect the Raspberry Pi with all the required cables and peripherals.

Student Task 3: The goal of this task is to connect the Raspberry Pi. A photo of the complete setup can be seen in Figure 4.

- Insert the previously flashed micro SD-card into the Raspberry Pi.
- Connect the Raspberry Pi AI camera to the Raspberry Pi Cam0 slot.
- Connect the network cable from the Raspberry Pi to your computer, using the provided USB to Ethernet adapter.
- Connect the power supply to the Raspberry Pi and to the power socket. The Raspberry Pi now boots up.

3.1.3 Setup of Network Interface

Now we will set the network interface of our computer to connect with the Raspberry Pi.

Student Task 4: The goal of this task is to get network connectivity to the Raspberry Pi.

- Connect your computer to a HotSpot created by your mobile phone. This step is necessary, as the ETH WiFi does not allow network sharing, which is necessary to connect the Raspberry Pi to the internet over your computer. If you aren't able to create a HotSpot by your mobile phone, e.g. due to a limited data plan, reach out to the teaching assistants.
- Setup the network adapter. For Windows, follow the steps in Figure 5. For macOS, follow the steps in Figure 6. If you use another operating system, find out how to do network sharing. You can ask the teaching assistants for help on this.
- Check the connection to the Raspberry Pi. For this open a terminal and enter `ping mlonuc`. You can see successful ping attempts in Figure 7. If the ping fails, recheck your network adapter settings.

3.1.4 Connect by SSH and enable VNC

Now that we have established network connection to the Raspberry Pi, we can install VNC on the Raspberry Pi to connect to it with the graphical user interface.

Student Task 5: The goal of this task is to setup a VNC server on the Raspberry Pi.

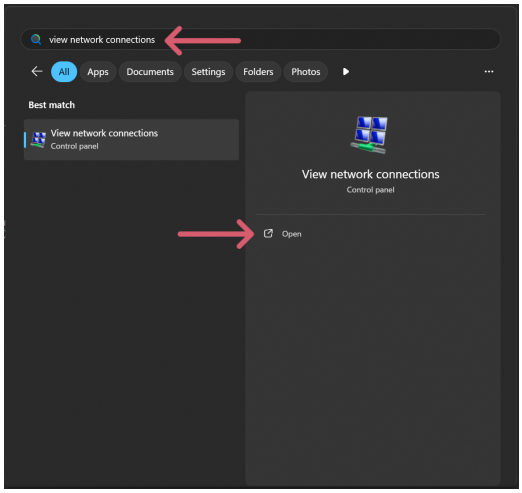
- Open a terminal and connect to the Raspberry Pi by SSH, the credentials are as set previously in Figure 2a: `ssh mlonuc@mlonuc.local`
- When connected by ssh to the Raspberry Pi, you can open the Raspberry Pi configuration with `sudo raspi-config`
- Now go to `Interface Options > VNC > Enabled: Set to Yes > Finish`
- Download RealVNC viewer from [a] and install it on your computer. We will use it to access the Raspberry Pi remotely.
- Now open RealVNC and in the top menu bar click `File > New Connection`
- As VNC Server, add the hostname you entered in Figure 2a, in our case `mlonuc`. Use the credentials as set in Figure 2a. The whole VNC connection process is explained in detail in Figure 8. Now you should see the GUI of the Raspberry Pi.

[a] https://www.realvnc.com/en/connect/download/viewer/

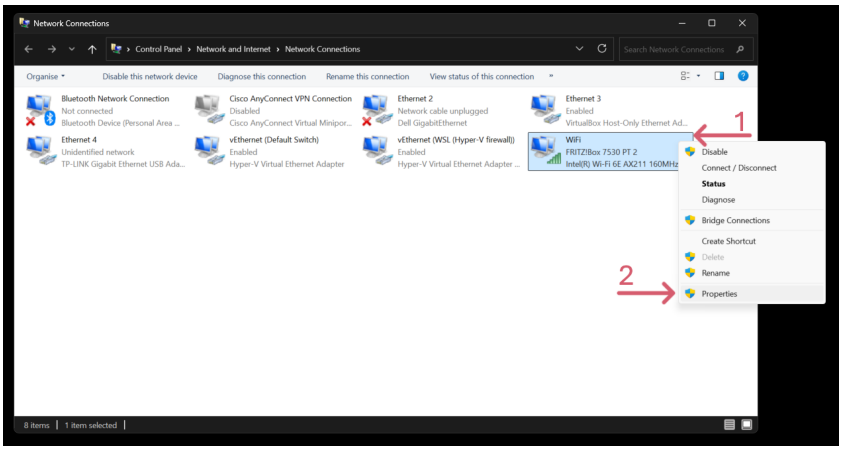
Now we can easily connect to the Raspberry Pi by VNC and thus use both the command line as well as the graphical user interface.

3.1.5 Updating and Installing Dependencies

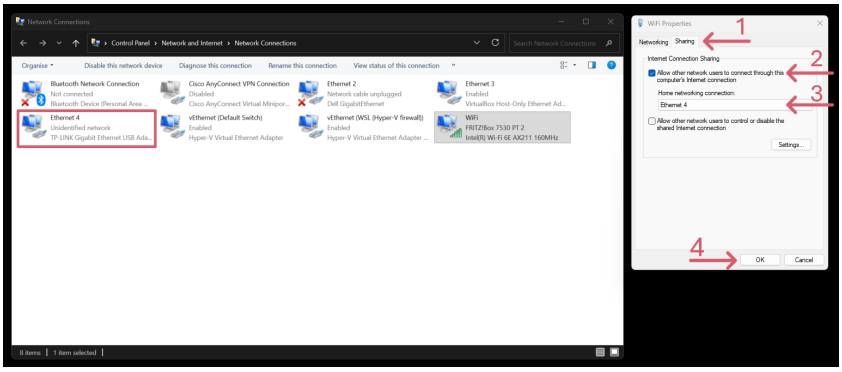
The Raspberry Pi has internet connectivity due to the link sharing of the network interface on your computer. As such, we can next update the pre-installed software as well as install the required dependencies.



(a) Open the `View network connections` settings which are part of the `Control panel`.

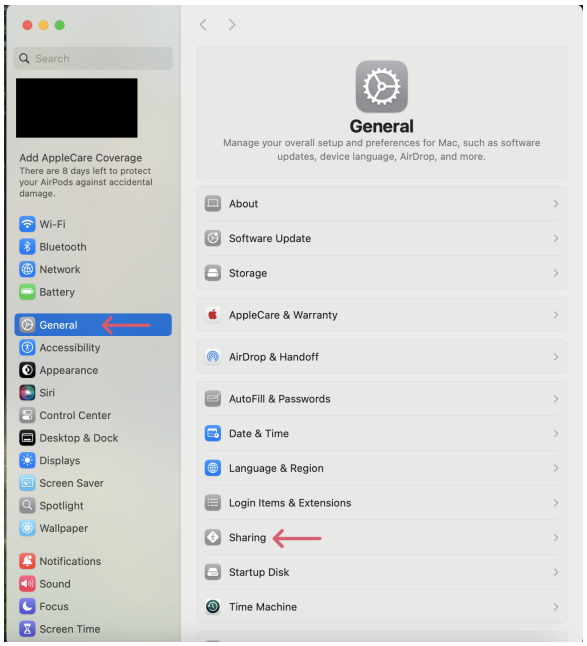


(b) Right click your WiFi connections and click on `Properties`.

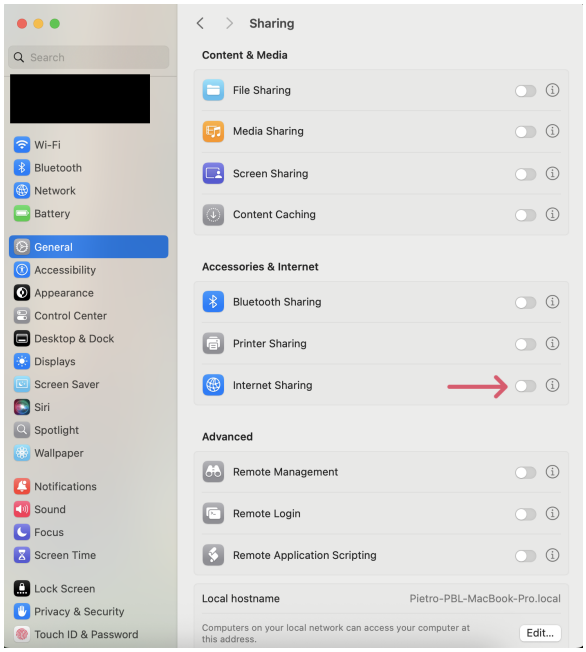


(c) Go to the `Sharing` tab, click on `Allow other network users to connect through this computer's internet connection` and select the correct `Home networking connection`. To know which one to select, look at which Ethernet connection uses the `TP-Link Gigabit Ethernet USB Adapter`, e.g `Ethernet 4` in this screenshot. Then click `OK`.

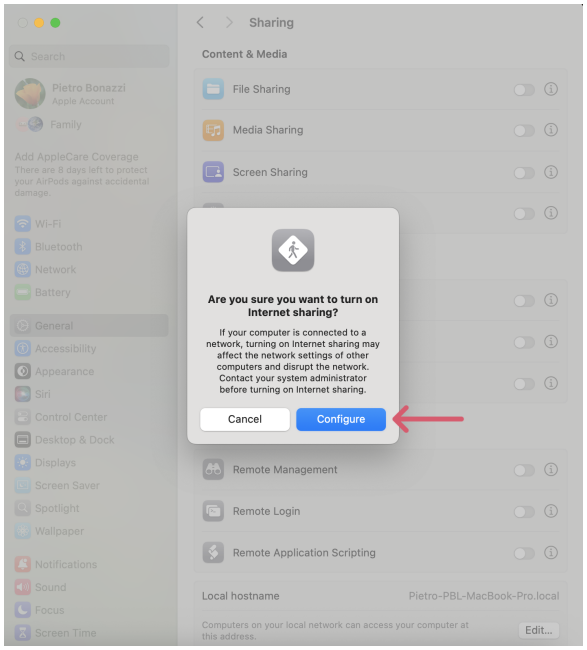
Figure 5: Windows network adapter settings



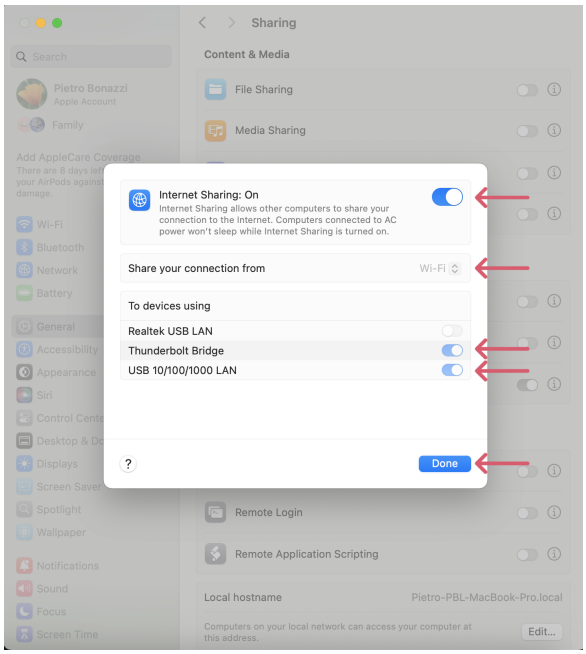
(a) Open `Settings` and go to `General`.



(b) Activate `Internet Sharing`.

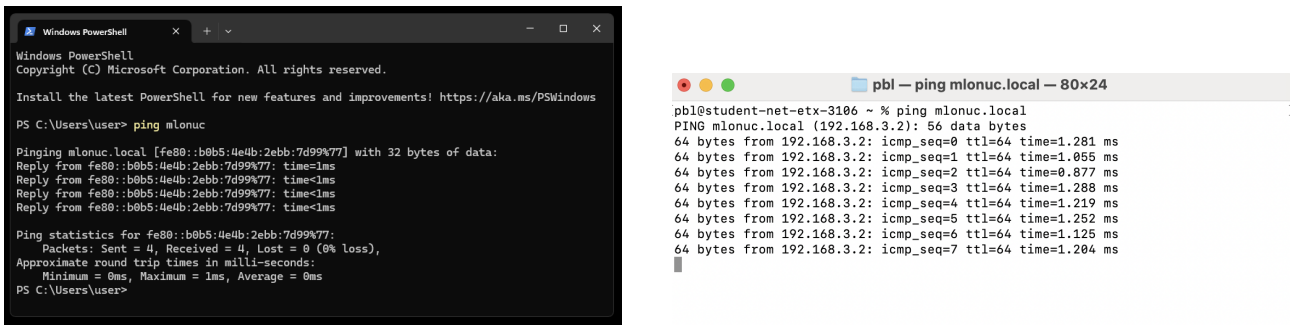


(c) Confirm by clicking on `Configure`.



(d) Activate the network sharing as in the screenshot and confirm by clicking on `Done`.

Figure 6: macOS network adapter settings.



(a) Successful ping on Windows.

(b) Successful ping on macOS.

Figure 7: Ping command to check connectivity. Note that depending on your shared network connection, it will either use IPv4 or IPv6 as seen in the screenshots, which doesn't have to worry us any further as we use the DNS name `mlonuc` and don't have to know the IP address of the Raspberry Pi.

Student Task 6: The goal of this task is to update and install software on the Raspberry Pi.

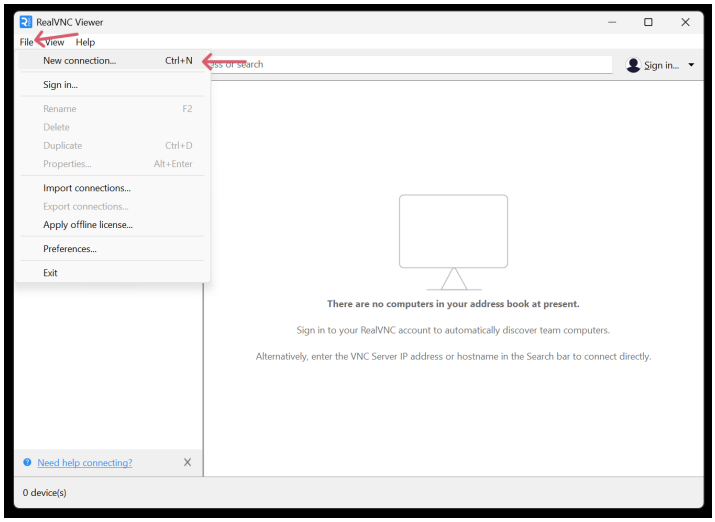
- Open a terminal on the Raspberry Pi.
- Update package repositories and update packages:
  `sudo apt update && sudo apt full-upgrade`
  Enter the previously set password when prompted to do so.
- Install the required IMX500 dependencies:
  `sudo apt install python3-opencv python3-munkres imx500-all`
  Enter the previously set password when prompted to do so.
- Later we will need a demo python application to run our model. Upload the file `imx500_classification_dem.py` to the Raspberry Pi using the following command:
  ```
  scp imx500_classification_demo_adapted.py \
  mlonuc@mlonuc.local:Desktop/mlonuc/imx500_classification_demo_adapted.py
  ```

3.1.6 Testing the Raspberry Pi Setup

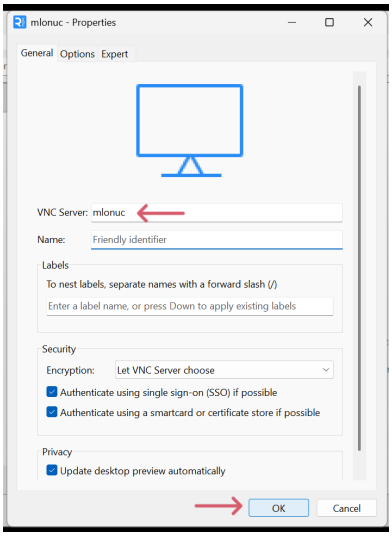
Now we have a running Raspberry Pi with all dependencies installed. As such it is time to test the setup including the camera functionality.

Student Task 7: The goal of this task is to connect and start the Raspberry Pi.

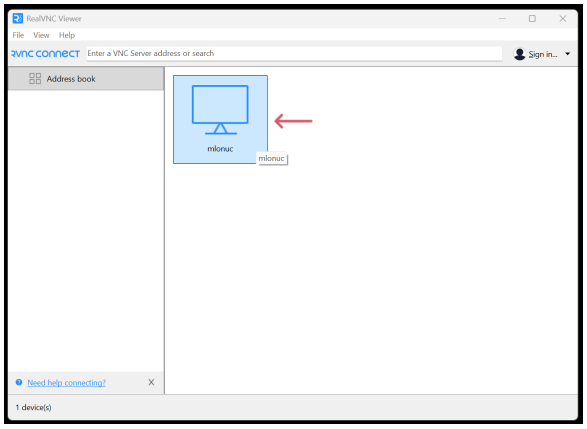
- Verify the functionality of the camera by executing
  ```
  rpicam-hello -t 0s --post-process-file \\
  /usr/share/rpi-camera-assets/imx500_mobilenet_ssd.json \\
  --viewfinder-width 1920 --viewfinder-height 1080 --framerate 30
  ```
  This command is also available on the Raspberry Pi in the file `README.txt` on the Desktop, as such you can easily copy it even when copying over VNC doesn't work. You should see a window showing the camera image live, that detects and classifies certain objects as well as persons. If an error occurs, make sure that the camera is connected properly and restart the camera with above command. If this doesn't help and the error persists, ask an assistant.



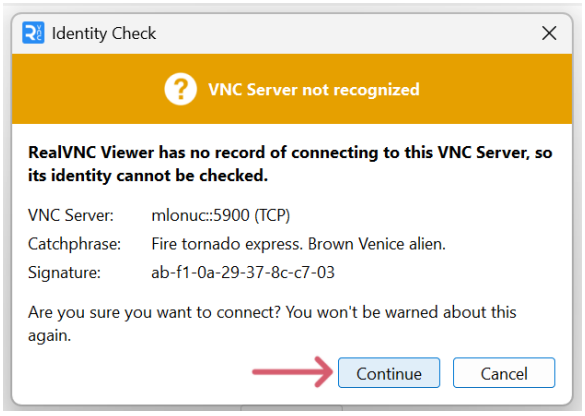
(a) Click on `File > New connection`.



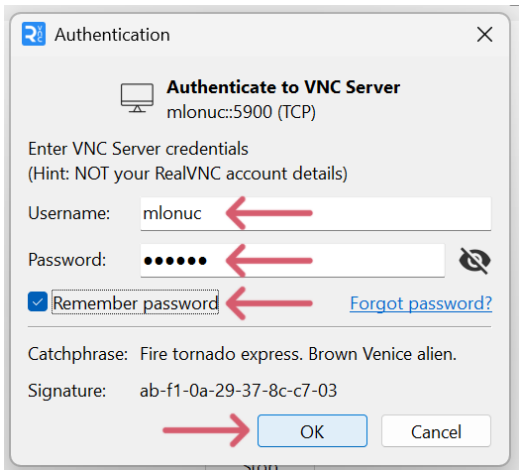
(b) As `VNC Server` enter `mlonuc` and click `OK`.



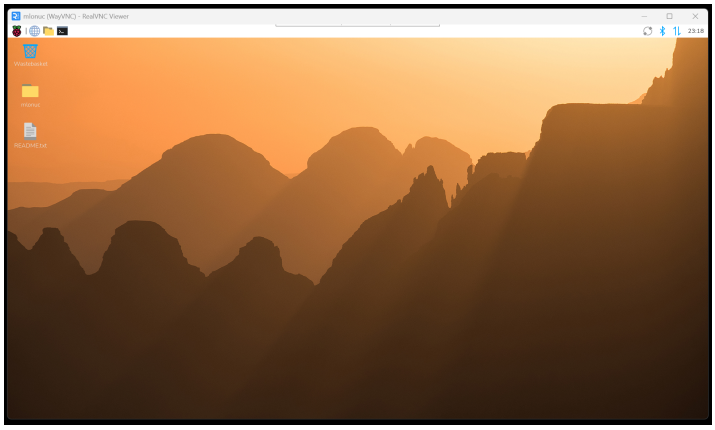
(c) Double click on `mlonuc`.



(d) Click on `Continue`.



(e) Enter `Username` and `Password`, enable `Remember password` and click on `OK`.



(f) If everything is correct, you will see the GUI of the Raspberry Pi.

Figure 8: Setup of VNC connection.

Measure	Unit	Value
Floating Point Accuracy	%	
Quantized Accuracy	%	
# of Parameters	Parameters	
# of MAC-operations	MACs	
Inference Time	ms	
# of clock cycles per inference	cycles / inference	
# of MAC per Cycle	MACs / cycle	
Energy per Image Acquisition	mJ	
Energy per Inference Measurement	mJ	
Energy per Image Acquisition Corrected by Idle Power	mJ	
Energy per Inference Corrected by Idle Power	mJ	
Model Memory Usage on IMX500	KB	
Total Memory Usage on IMX500	KB	
Memory Utilization on IMX500	%	

Table 1: Table for your results.

If above test of the camera was successfully, you have now a working Raspberry Pi setup.

3.2 Training environment on Google Colab

For this exercise we will use Google Colab. This allows us to use their GPU for free, which will reduce training time greatly.

Student Task 8: The goal of this task is to login to Google Colab and prepare the training environment.

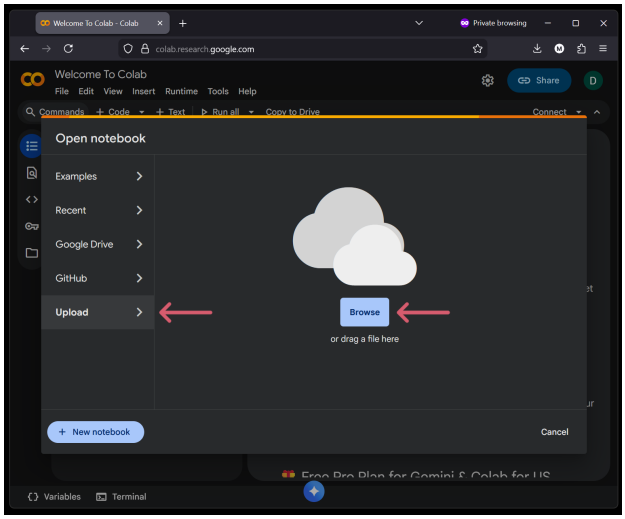
- Login to Google Colab[a] using any Google account.
- Follow the setup guide in Figure 9.
- Leave the Google Colab window open for later.

[a] https://colab.research.google.com

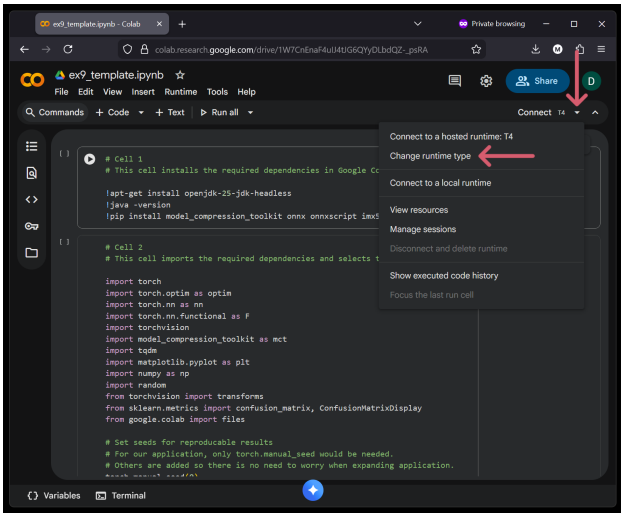
4 Model Metrics

To compare the performance of our model to others, we need to evaluate different metrics. In this exercise, we will guide you through this step by step.

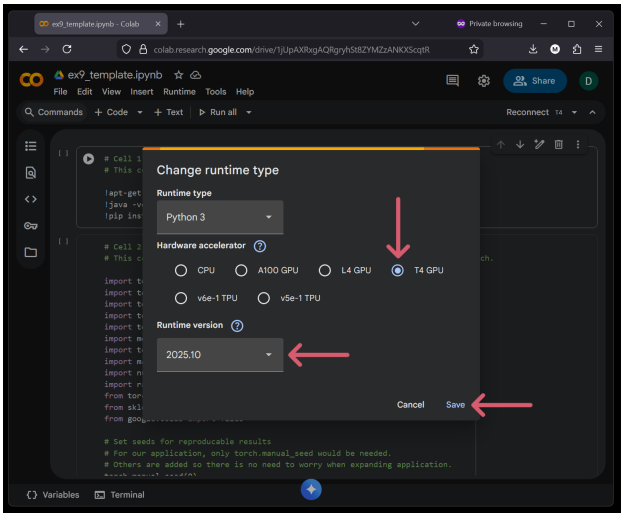
Write the results into Table 1 when asked to do so.



(a) Select `Upload > Browse` to upload the provided Jupyter Notebook `ex9_template.ipynb`.

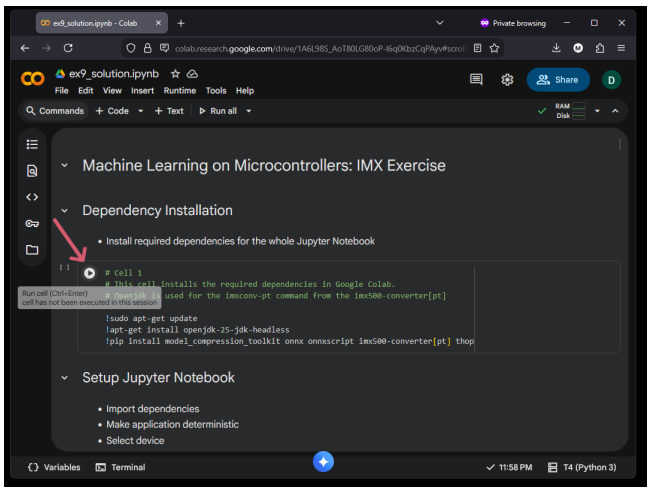


(b) Click on the marked down arrow and then on `Change runtime type`.

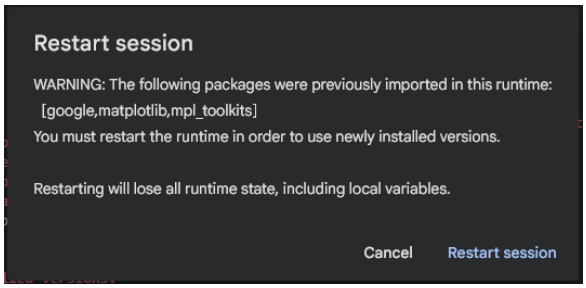


(c) If available, choose a `T4 GPU` unit, otherwise select `CPU`. Select the `2025.10 Runtime version` and then click on `Save`.

Figure 9: Google Colab setup.



(a) Execute the first cell by clicking on the play button.



(b) Restart the session after cell 1 has executed, as asked by the popup window.

Figure 10: Execution of cell 1.

5 Training, Validation and Conversion

Now we will need to create our model for the classification task, train it, validate it, quantize it and convert it to be able to run it on the Raspberry Pi. Note that this exercise uses PyTorch, but for the IMX500 also the usage of TensorFlow would be possible.

5.1 Start Execution

Before we start with the interesting part, we need to install the required dependencies. The first cell in the Jupyter Notebook contains all required commands to install the dependencies.

Student Task 9: The goal of this task is to start the execution of the Jupyter notebook and install all required dependencies.

- Start cell 1 by clicking on the play button as seen in Figure 10a. This will install the required dependencies.
- After the first cell has finished running, a popup window as in Figure 10b. Click `Restart session`.
- Further, the installation of the dependencies will give us some errors, as seen in Figure 11. In this case, we can ignore the error. In practice, always make sure to understand any errors or warnings to ensure correct behaviour of your software.
- Now you can run cell 2. This cell imports all required python libraries. Further it makes our results reproducable and also chooses the device to run our code on. If a GPU is available it will run on the GPU, otherwise it will run on the CPU.

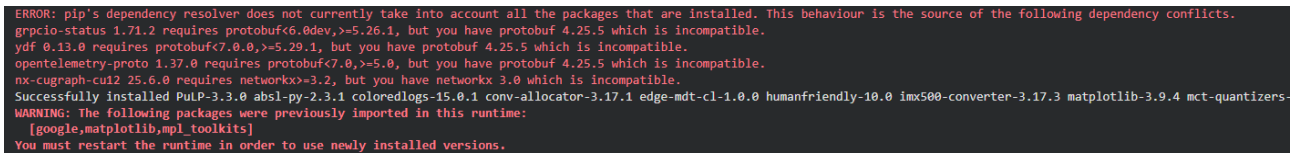


Figure 11: This error can be ignored for this exercise.

5.2 Basic Understanding of the ResNet Model

There is a variety of models which can be used for CIFAR-10 classification. In this exercise, we use a model with residual connections, which is inspired by ResNet[3]. Residual connections, mathematically expressed as $y = f(x) + x$ allow us to mitigate vanishing gradients in large models. It can also be viewed as retaining the input information by letting it go trough. Now lets go on to explore the provided model.

Student Task 10: The goal of this task is to get familiar with the provided model. For this, look at the ResNet class in the provided python code and answer the following questions.

- What do the three input layers of the model correspond to?

  _____

- What types of layers are used in the model?

  _____

- What types of activation functions are used in the model?

  _____

- How are the residual connections implemented?

  _____

- How many blocks are there per layer?

  _____

Note: It is important to note that the provided model is in no way specifically optimized to our classification task or hardware. It is a generic model architecture which does reasonably well on the CIFAR-10 dataset. It is used as a illustration and you can use it as a foundation to create your own optimized models from it.

[3] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep Residual Learning for Image Recognition', arXiv [cs.CV]. 2015.

5.3 The Model Output

In classification tasks, the output vector is often normalized to sum up to 1, while each element of the output vector is in $[0, 1]$. As such, we can interpret the output vector as a probability distribution, e.g. each output element represents the probability that the input belongs to that class. An often used function for this normalization is softmax. Note that adding this normalization can also be interpreted as adding an activation function at the output of the network.

Student Task 11: The goal of this task is to add softmax normalization to the models output and sharpen the understanding on what the output of the model represents.

- Rescale the outputs of the given neural network using softmax. Your code should be added at `TODO1` and `TODO2` in cell 3. For more information about the PyTorch softmax function, see [a].

- What is a potential drawback of using a 10-dimensional output vector (one output node per class) to classify among 10 categories? Hint: What does the model do when it is given an input that does not belong to any of the 10 classes?

- Now run cell 3. The test will show the output shape. Is this the desired output shape?

---

[a] https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html

5.4 Analyzing Model Key Metrics

The number of parameters counts all weight parameters the model has. This directly translates into memory requirements, as each parameter has to be stored somewhere. The exact storage cost also depends on the data type used: for example, a single-precision floating-point parameter (float32) typically requires 4 bytes, whereas an 8-bit integer parameter needs only 1 byte.

The provided code in PyTorch prints both the number of parameters per layer as well as of the whole model. Nonetheless, it can be beneficial to be able to compute the number of parameters oneself, e.g. to understand how the parameter count scales by changing different settings in the model.

Multiply–Accumulate (MAC) operations are a standard way to estimate the computational cost of a machine learning model. A single MAC consists of one multiplication followed by one addition, which is exactly what occurs in common neural network layers such as fully connected and convolutional layers. MAC counts provide a simple metric that correlates with latency, energy consumption, and computational requirements, making them useful for comparing different models. However, MACs alone offer only a partial view of computational efficiency. Actual runtime performance also depends

heavily on model structure, such as the degree of parallelism, memory access patterns, and data movement costs. Therefore, MAC counts should be treated as an estimate rather than a complete measure of the effort required for inference.

Student Task 12:

- Calculate the number of parameters of the initial convolutional layer. Hint: How many input channels do we have and how many output channels? What is the kernel size? How are these metrics related to each other. Drawing a picture can help. Note that we don't have a bias term for the initial layer. If you are unsure how to get to the result, talk with an assistant.

- How much memory is required to store the initial convolutional layer's parameters using float32 versus int8?

- Calculate the number of parameters of the initial batch normalization layer.

- Now run cell 4.
- Read out the number of parameters of the initial convolutional layer from the output of cell 4. Does it match your calculation?

- Read out the number of parameters of the initial batch normalization layer from the output of cell 4. Does it match your calculation?

- Read out total number of parameters from the output of cell 4 and write it into Table 1.

- Read out the total number of MAC operations from the output of cell 4. Write it into Table 1.

5.5 Visualizing the Model

Now we want to visualize the model to better see how it is structured. For this we will export the model in the Open Neural Network Exchange (ONNX) format and then open it in Netron[4].

Student Task 13: The goal of this task is to export the model in the ONNX format and inspect it in Netron.

- Now run cell 5. The ONNX file will be automatically downloaded to your machine as `floating_point_model_untrained.onnx`.
- Open the downloaded file in Netron [a].
- Study the network graph. Can you see the residual connections? Is the network structured as you would expect it from the previous code analysis?

- Leave the window with Netron open, as we will use it again later.

[a] https://netron.app/

5.6 On Data Transformations

Data transformations are a universal tool used in machine learning. They can be used for different purposes. In this project we use data transformations to resize the training and test data to be compatible with the hardware module. Further we use transformations for the common data augmentation technique.

Lets first focus on the hardware requirements. As noted in the key specifications [5] of the IMX500, the input tensor size has size requirements as follows:

- Minimum input size: 64(H)x48(H)
- Maximum input size: 640(H)x480(H).

As CIFAR-10 only has size 32x32, we need to upscale the input size accordingly. For this, PyTorch offers the `Resize` transformation.

Data augmentation is used in machine learning to artificially increase the quantity and diversity of training data, making models more robust and better at generalizing. This allows our network to become more robust to this kind of transformations. PyTorch for example offers `RandomHorizontalFlip` and `RandomAffine` transformations for this use case. Note that not for all problems all those transformations are applicable.

[4] https://netron.app/
[5] https://developer.sony.com/imx500/imx500-key-specifications

Student Task 14: The goal of this task is to add the required and useful data transformations. Use the official documentation[a].

- Add a `Resize` transformation with output size 64x64 pixels to account for the minimum input size of the processing unit. Do this for both the training and test data, the code can be found at `TODO3` and `TODO4` respectively.

- Add a `RandomHorizontalFlip` transformation to augment the training data. This transformation randomly flips the image horizontally during training. Do this only at `TODO3`.

- Add a `RandomAffine` transformation, again to augment the training data. Do this only at `TODO3`. In practice, one has to try different parameters and choose good parameters using a validation data set, e.g. doing k-fold cross validation. This is called hyper parameter selection. For your project, it is highly recommended to do hyper parameter optimization. For now, you can use: degree = 5, translate = (0, 0.05), scale=(0.95, 1.05), shear = 10. Again, only do this at `TODO3`.

- Which of the above data transformations are required to be able to correctly run our model later? Which are optional?

- Why do we not apply `HorizontalFlip` or `RandomAffine` transformations on our test data?

- Which other transformations could you add for data augmentation?

- Now run cell 6.

[a] https://docs.pytorch.org/vision/main/transforms.html

5.7 Importing the Dataset

To train our model, we first need to import the data set. A lot of libraries offer integration for the most common data sets, as such no manual import functionality has to be written. Luckily, PyTorch offers the integration of the CIFAR-10 dataset.

Additional to the data, our model requires the number of classes to classify into in order to set the output vector to the correct size. Further we would like to have a list of text labels that correspond to our classes. The output of the model is simply a vector, and the probability that the input belongs to a specific class is an element of said vector. Using the text labels, we can later assign meaningful names to the elements of the output vector, especially when plotting our results. Further we can visualize our data. Visualizations often help to better understand the problem we are trying to solve. Further we can visualize some samples from the data augmentation and see how it affects the data.

Note: The batch size is also defined in cell 5. Note that batch size influence the training greatly, but is not discussed in this exercise. Further, for efficient training the batch size has to be chosen such that a whole batch can be loaded onto the GPU at the same time.

Student Task 15: The goal of this task is to understand the import of the CIFAR-10 training and test data and to further visualize the imported data.

- Look at the provided code that imports the dataset in cell 7. You see the `batch_size` as well as the download of the dataset and the instantiation of the `DataLoader` for both the training as well as the test data. Now run cell 7.
- Look at the provided code in cell 8. It defines the number of classes for our problem, here. 10 because CIFAR-10 has 10 classes. Further it initializes a list of the class labels in a human readable form. For more information about CIFAR-10, see [a]. Now run cell 8.
- In cell 9 we visualize 4 samples per class from both the test as well as the training dataset. Run cell 9. Can you see the effect the data augmentation has on the training data?

[a] https://www.cs.toronto.edu/~kriz/cifar.html

5.8 Training the Model

To train the model, we need a criteria, also called loss. Further we also need an optimizer. Typically, a cross entropy loss is used for classification tasks, which is also available in PyTorch[6]. As an optimizer, we can for example use Adam[7] or stochastic gradient descent (SGD).

When training for a lot of iterations, it is possible to achieve the best model performance in terms of test accuracy in an earlier iterations than the last one. This can for example be due to overfitting. As a solution to this problem, we can calculate the test accuracy after each iteration. If our new model is better than the old one, we replace the old one. Otherwise we keep the old one.

Student Task 16: The goal of this task is understand potential issues with cross entropy loss.

- Look at the code in cell 10 to understand how to use `CrossEntropyLoss` and `Adam` optimizer in PyTorch
- What could be an issue with `CrossEntropyLoss`? Is it an issue with the CIFAR-10 dataset?

[6] https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html
[7] https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html

Hint: Look how many training samples are available of each class and look at the documentation[a].

- If you use a GPU instance in Google Colab, now run cell 11. This will train the model and take approximately 17 minutes. In this time, you can do subsection 3.1, which is independent of Google Colab. If you are finished with this, have a look at the this paper about the IMX500[b]. If only a CPU instance is available, you can upload `best_fp_model.pth` to Google Colab.
- Run cell 12 which either loads the best model from the training or the model you uploaded.

[a] https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[b] R. Eki et al., "9.6 A 1/2.3inch 12.3Mpixel with On-Chip 4.97TOPS/W CNN Processor Back-Illuminated Stacked CMOS Image Sensor," 2021 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 154-156, doi: 10.1109/ISSCC42613.2021.9365965. https://ieeexplore.ieee.org/document/9365965

5.9 Evaluation of the Training Process and the Model

At this point we have a model that was trained for 10 epochs, using some basic tricks such as data augmentation. But are the number of epochs enough? Is the model overfitting or underfitting? Does the model perform according to our requirements? These are some questions that we have to ask ourselves when training a model.

Test accuracy is one of the key characteristics of a model. It directly tells us how good the model performs on a test dataset. It represents the fraction of predictions that are correct on a test set, where the test set necessarily has to be disjoint from the training set.

Test accuracy and loss however only have a limited view on the result, as we only see a global view. To better understand our model, we can use a confusion matrix. There exists complete implementation of this, for example in sklearn [8] or one can implement it by oneself with a standard ploting library like matplotlib.

Student Task 17: The goal of this task is to analyze our model training.

- What is the test accuracy of our best floating point model? Write it into Table 1.

- If you trained the model yourself, which means you did not upload the file `best_fp_model.pth`, now run cell 13 and look at the graphs. Did training finish completely? Should we train for more iterations? If you didn't train the model yourself, answer the questions by looking at Figure 12.

[8] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixDisplay.html

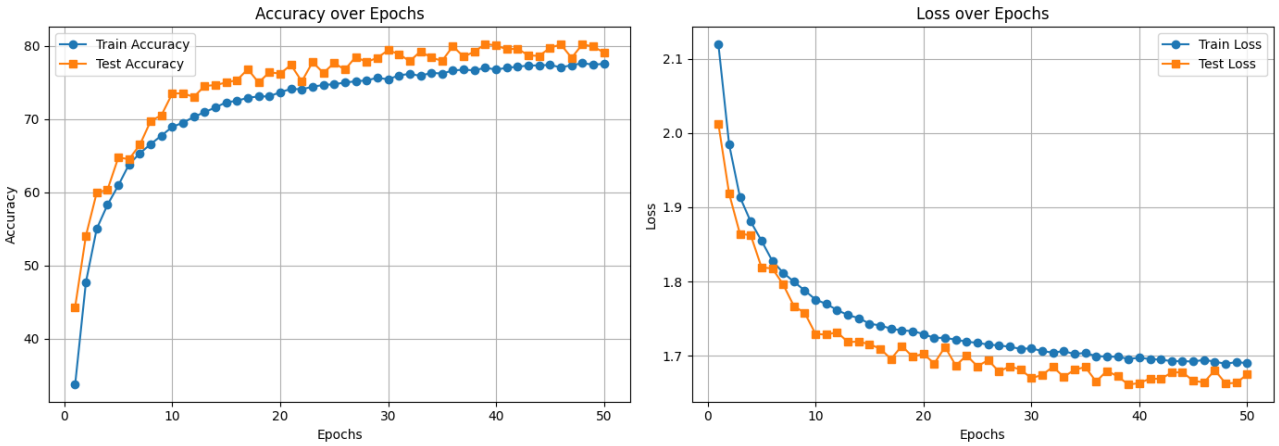


Figure 12: Training and test accuracy as well as loss for 50 epochs.

- Run cell 14. Then study the confusion matrix. What are the key take aways from the confusion matrix?

- What could we do to get better on the classes that are predicted poorly?

5.10 Model Quantization and Conversion for IMX500

Now as we have a working floating point model, we need to quantize it for the IMX500 and convert it to the corresponding representation to load it onto the hardware. For this we use the Sony model compression toolkit [9]. The steps that we need to do are:

- Post training quantization using the IMX toolkit
- Testing the accuracy of the quantized model
- Storing the model in ONNX format for further processing
- Convert the ONNX format file to an IMX specific representation using the imxconverter

A complete overview of the work flow for the IMX500 can be found in the documentation[10].

The post training quantization config offers a variety of parameters that can be changed. For example, the `activation_error_method` and `weights_error_method` can be chosen from a variety of

[9] https://github.com/SonySemiconductorSolutions/mct-model-optimization
[10] https://developer.aitrios.sony-semicon.com/en/docs/raspberry-pi-ai-camera/imx500-packager?version=2025-09-30&progLang=

functions[11]. This is part of the design space, where the mean square loss (MSE) is an often used default metric. However, better results can be achieved by using an appropriate error function for a specific application.

Sadly, the official documentation[12] isn't that extensive to get to know the functionality of the model compression toolkit. However, on the GitHub readme[13], a lot of Google Colab notebooks are linked with very good examples and detailed descriptions.

The IMX model compression toolkit also offers other possibilities to quantize the model, such as quantization aware training and gradient post training quantization. Post training quantization is the simplest one, as such this introductory exercise uses it. However, feel free to explore the other possibilities in your project.

Further it is possible to export the ONNX file after the quantization step. This allows us to analyze the quantized model and compare it to the floating point model.

Student Task 18: The goal of this task is to quantize the floating point model and convert it into the required representation to load it onto the hardware.

- Look at the code in cell 15. You can find out more about the used functions in the official documentation or the example notebooks of the model compression toolkit. Now run cell 15.
- Run cell 16. What is the accuracy of the quantized model? Write it into Table 1.

- What effect did the quantization have on the accuracy?

- Run cell 17. This exports the quantized model in the ONNX format and will automatically download it to your machine. We will use it in subsection 5.11 to compare the quantized model to the floating point model.
- Run cell 18. This converts the ONNX file format to an IMX500 proprietary format. The resulting zip archive `imxconv_out.zip` containing the converted files will be downloaded automatically to your machine. We will use it later in section 6. In `memory_report.zip` a memory report and visualization of the resulting network are available.
- Look at `quantized_model_MemoryReport.json` inside `memory_report.zip`. How much memory do we use? How much is used for the runtime, e.g. the software running on the IMX500 chip, and how much is used for the model? How much total memory is available? Does the model fit in the chip?

[11] https://sonysemiconductorsolutions.github.io/mct-model-optimization/api/api_docs/classes/QuantizationErrorMethod.html
[12] https://sonysemiconductorsolutions.github.io/mct-model-optimization/api/api_docs/
[13] https://github.com/SonySemiconductorSolutions/mct-model-optimization/tree/main?tab=readme-ov-file

- Complement Table 1 with the applicable data from the previous step.

5.11 Visualizing the Quantized Model

At this point we want to compare our floating point model and our fixed point model in terms of how they are structured.

Student Task 19: The goal of this task is to visualize the quantized model and compare it to the floating point model.

- Open the ONNX file `quantized_model.onnx` that was downloaded to your machine in Netron.
- How is the quantization done?

- Why is no batch normalization visible?

- Now open `quantized_model.pbtxt` from the `memory_report.zip` in Netron.
- How is the quantization done?

- Is the general structure of our model still the same?

- To how many bits are the convolutional layers quantized to?

- To how many bits is the fully connected layer quantized to?

- To how many bits is the softmax operation quantized to?

---

6 Running the Model on the AI Camera

At this point we have a trained model, already quantized by the IMX model compression toolkit. What remains is to load it onto the Raspberry Pi, package it into a format that can be loaded on to the IMX500 and creating an example application to visualize our results.

6.1 Transferring the imxconv_out.zip Archive to the Raspberry Pi

Now we will need to load our quantized model folder onto the Raspberry Pi. For this we will use Secure Copy (scp)[14], an often used protocol for such tasks.

Student Task 20: The goal of this task is to transfer the imxconv_out.zip archive to the Raspberry Pi.

- Copy the previously downloaded imxconv_out.zip archive to the Raspberry Pi. Open a terminal on your computer in the folder, where the downloaded file resides. Then copy the file to the Raspberry Pi with the following command and enter the password when prompted: `scp ./imxconv_out.zip mlonuc@mlonuc.local:Desktop/mlonuc`

6.2 Packaging the Model File

Now the packer output file is on the Raspberry Pi. What is still missing is the conversion to the final file format that can be loaded onto the IMX500. This requires the IMX500 packaging tool, which was already preinstalled with the command `sudo apt install imx500-all`

Student Task 21: The goal of this task is to execute the IMX500 packager onto our IMX model compression toolkit output.

- Open a new terminal and change to the directory by executing `cd /home/mlonuc/Desktop/mlonuc`. The following command is again available in `README.txt` on the Desktop of the Raspberry Pi. Then execute the following commands:

```
unzip imxconv_out.zip
imx500-package -i imxconv_out/packerOut.zip -o imx500_tools_out
```

6.3 Running the Classification Application

To visualize our application, we will show the camera image and the classification output. For this we will use the example application from the Raspberry Pi Foundation[15]. The script was adapted to the CIFAR-10 dataset and is a good start for your own applications.

Student Task 22: The goal of this task is to run the model and the visualization of the results. The required file is already on the Raspberry Pi.

- Using the same terminal as before, run the following command to start the application. It is again available in `README.txt` on the Desktop of the Raspberry Pi.

[14] https://de.wikipedia.org/wiki/Secure_Copy
[15] https://github.com/raspberrypi/picamera2/tree/main/examples/imx500

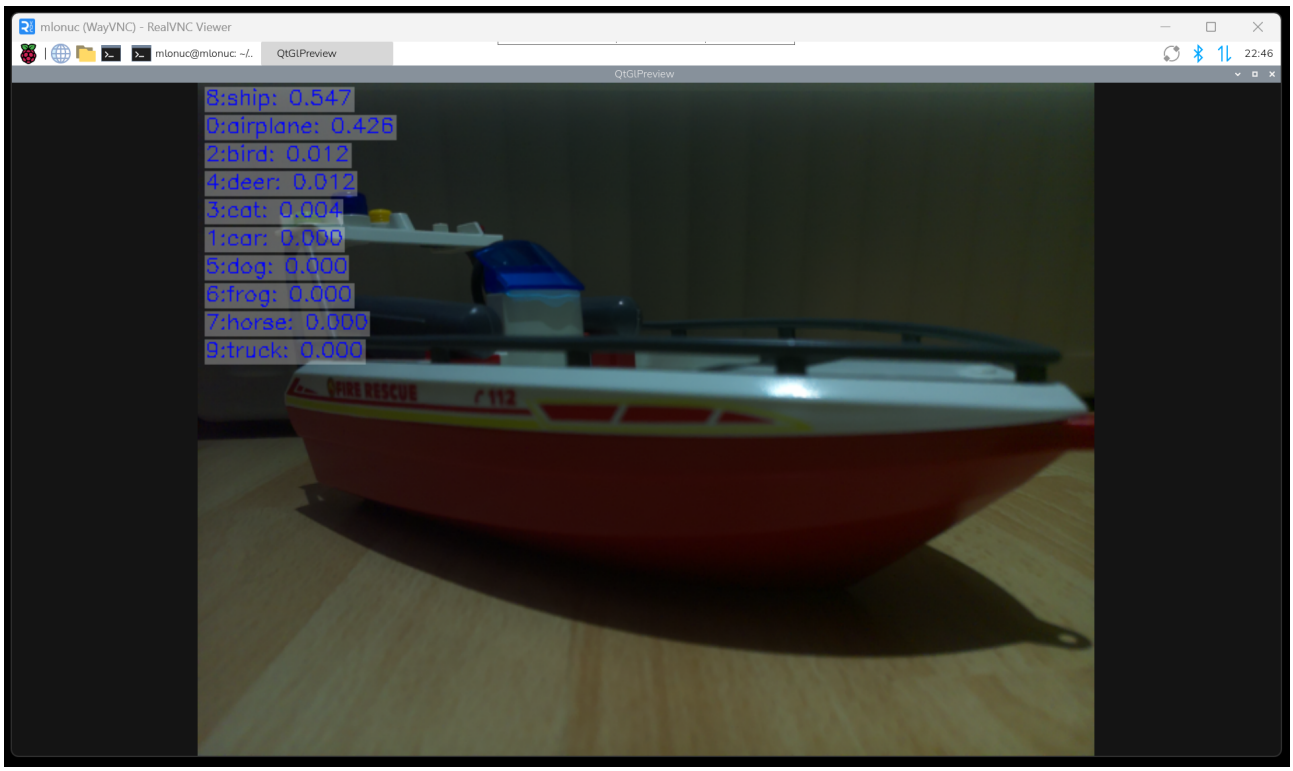


Figure 13: The running application, showing the camera image as well as the inference results.

```
python imx500_classification_demo_adapted.py --model \
imx500\_tools\_out/network.rpk --labels assets/cifar10_labels.txt
```

A window with the camera image and the output of the IMX500 will appear as in Figure 13.

- Test your application, e.g. by opening images on your mobile phone and pointing the camera onto it. Does it work correctly?

6.4 Analysis on the Running Model

Now as we have a running inference, we will further analyze our quantized model.

Student Task 23: The goal of this task is to calculate further key metrics of our quantized model using both the results from the packaging tool as well as data from the paper about the IMX500[a].

- When running the classification application as in subsection 6.3, the terminal will output a DNN and a DSP runtime. Restart the application multiple times and compare the values. What happens? Sadly, we do not have any data from Sony what the DNN and the DSP runtimes are. In section 7 you will see what it likely refers to.

- Estimate the number of clock cycles. For this use the DNN time and the clock frequency from Figure 9.6.5 in the IMX500 paper.

- Estimate the number of MAC operations per cycle from the number of clock cycles previously computed and the number of MAC operations you previously wrote into Table 1. Write your result into Table 1.

- How can you check if your MAC per cycle result is reasonable? Is your result for MAC per cycle reasonable?

---

[a] R. Eki et al., "9.6 A 1/2.3inch 12.3Mpixel with On-Chip 4.97TOPS/W CNN Processor Back-Illuminated Stacked CMOS Image Sensor," 2021 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 154-156, doi: 10.1109/ISSCC42613.2021.9365965. https://ieeexplore.ieee.org/document/9365965

7 Measurement of Inference Time, Power and Energy

An accurate power number can be obtained by measuring the power. As the camera sensor and processor are on the same chip, it is quite difficult to measure separate power in an easy way. Instead, we focus on measuring the power of the whole AI camera over time, then separate the image acquisition and inference steps. Due to the constant sampling rate of our power measurement, we can simply calculate the average in each of those section, multiply by time and get our energy per image acquisition and energy per inference respectively.

To measure the power, a shunt resistor is used. The resistor is placed in the supply line of the camera. For this, a camera cable is modified. The supply line is cut trough and the copper on both sides is carefully uninsulated. Then two wires are soldered onto the copper planes. Between those two wires, a shunt resistor is placed. On this shunt resistor, the voltage is measured with two oscilloscope channels. The circuit is shown in Figure 14 and the setup is shown in Figure 15. From the measured voltages, we can calculate the current. as follows

$$I = \frac{V_{in} - V_{out}}{R_{shunt}}. \tag{1}$$

From this, we can calculate the power

$$P = I \cdot V_{out}. \tag{2}$$

Our measurement is time discrete, as such we can calculate the power for each sample separately. With our known sampling time, we can easily calculate the energy

$$E = P \cdot t \approx \sum_i P_i \cdot \Delta t = \Delta t \sum_i P_i, \tag{3}$$

where P_i is the power calculated with above formula at sample i.

Note that the shunt will influence the power measurement, as the shunt itself reduces the voltage by the dropout across it. A too large shunt resistor will even destroy our measurement completely, as the circuit can't run due to it. Using a too small shunt resistor the voltage across it is difficult to measure and susceptible to measurement noise.

When doing the measurement for our model, a graph as in Figure 16 will result. For this, an Analog Discovery 2 USB oscilloscope was used. A math channel was used to directly calculate the power, from the voltages. Figure 17 shows the corresponding voltages, where channel 1 is V_{in} and channel 2 in V_{out}.

In Figure 17 you see a hypothesis about what the DNN and DSP runtimes are. According to the measurements, it is possible that the DNN runtime is the inference time for the whole network, including some preparation and finalization steps with lower power. The DSP runtime is compute heavy, with a much higher power and is the neural network inference itself. Note tat this remains speculations based on the measurement results.

Student Task 24: The goal of this task is to measure the inference time as well as the energy for the inference and the energy for the image acquisition from a real measurement. The measured V_{in} and V_{out} are available as both a graph as well as in the provided `MeasurementDataTemplate.xlsx`. The R_{shunt} is 1.2 Ohm

- Calculate MAC per Watt with the power given in Figure 16. How representative is this number?

- Read out the image acquisition and inference energy from Figure 16. Write the results into Table 1.

- Which energies do these numbers represent? Is it really the energy used for the image acquisition and the inference respectively?

- Modify the provided `measurement.xlsx` file to calculate both the energy for the image acquisition as well as for the inference. Implement Equation 1, Equation 2 and Equation 3 for all samples. Only calculate the energy per sample, not yet the summed up energy of all samples. You need to find out where the acquisition and inference starts and ends. For this, read out a suitable threshold from the diagram. Then highlight the results in channel 2 that fulfill this threshold. Now sum up the highlighted samples for the image acquisition and the inference. Hint: You can implement the equations per sample and sum up the energy. Δt can also be calculated from the provided data. Note: Of course such data analysis can also be automated by a script, e.g. in Python. However, for our requirements, the excel sheet does very well.

- How do the results between the excel calculation and your estimation from the diagram match?

- How could we improve our results to better represent the power used for the image acquisition and the inference? Calculate an improved energy per image acquisition and energy per inference and write it into Table 1.

- When we don't have the equipment to do measurements as we have it in this exercise, how can we get a simple approximation of the inference energy?

Note: Our results show that model inference accounts for only a small share of total energy use. The Raspberry Pi AI camera already draws about 250 mW "at idle" and image acquisition consumes more power over a much longer period than inference does. It would be useful to determine what this 250 mW is actually used for. It is likely dominated by continuously transmitting image data to the Raspberry Pi, meaning it isn't truly idle power but transmission power. Disabling image transmission and measuring the power again could confirm this. If most of the 250 mW is indeed due to transmission, the AI camera and inference on it could provide substantial energy savings over doing inference on the Raspberry Pi itself using a normal camera, as long as the image is not required on the Raspberry Pi. This would be a very interesting aspect for you to discover in your project.

8 Conclusions and Ideas for Further Work

In this exercise, you learned the complete workflow from training a model to running it with the IMX500. It is important to emphasize that the various settings used in this exercise, e.g. hyper-

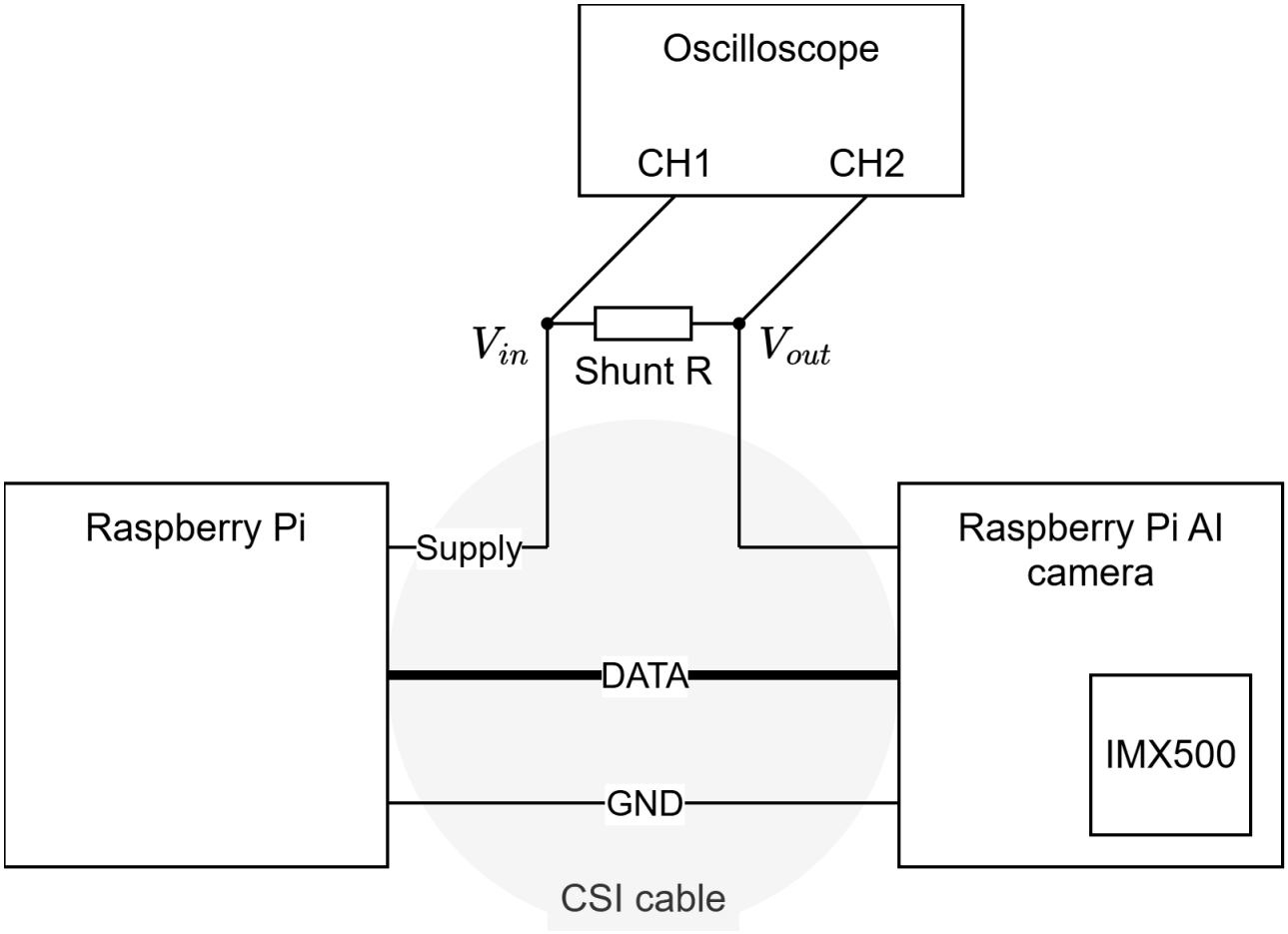


Figure 14: Measurement setup. The shunt is placed in the supply line between the Raspberry Pi and the Raspberry Pi AI camera.

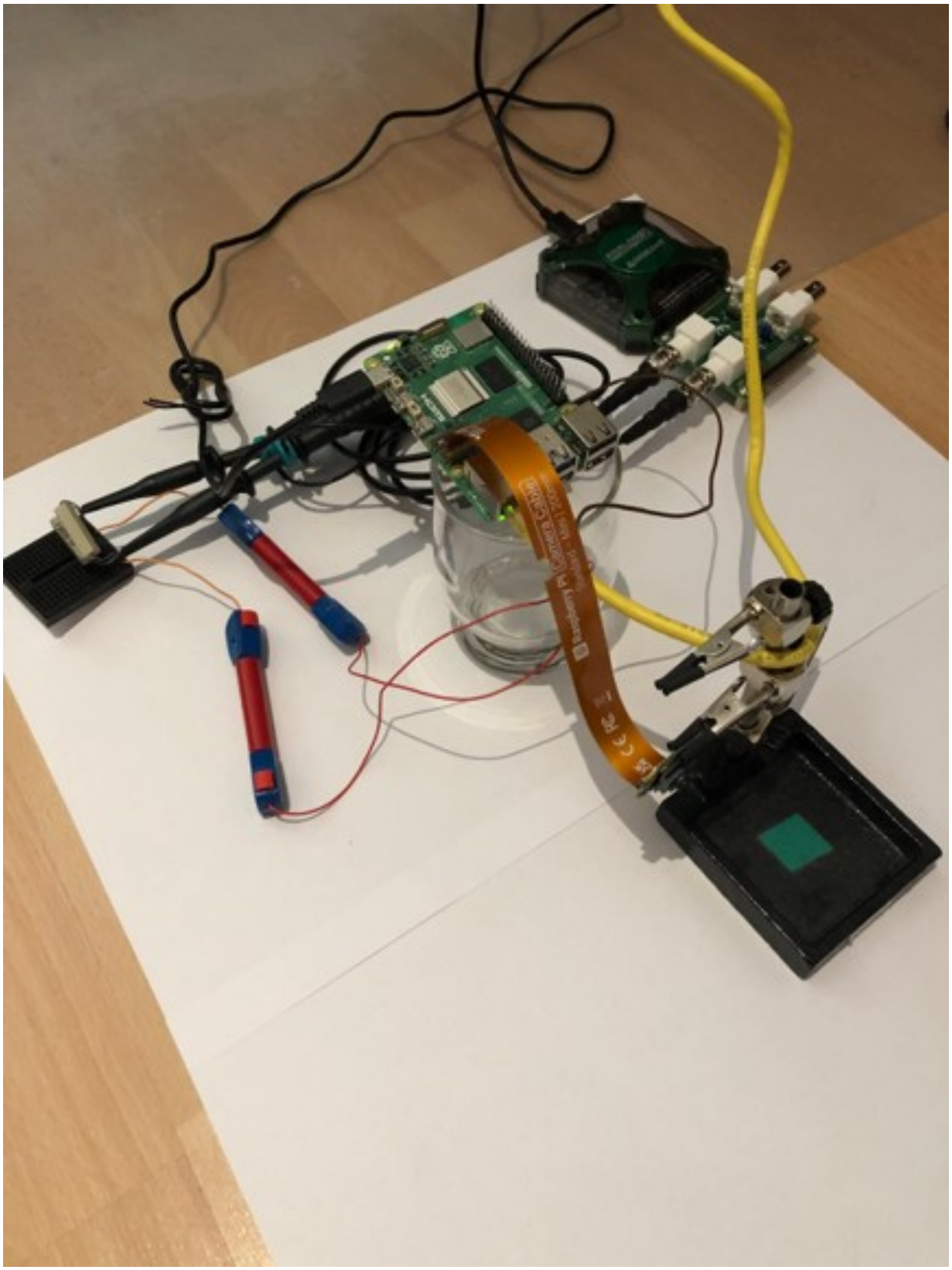


Figure 15: Measurement setup using an Analog Discovery 2 UBS oscilloscope. Note that in terms of measurement noise, the setup with the long cables is not optimal. However it is good enough for an approximate measurement.

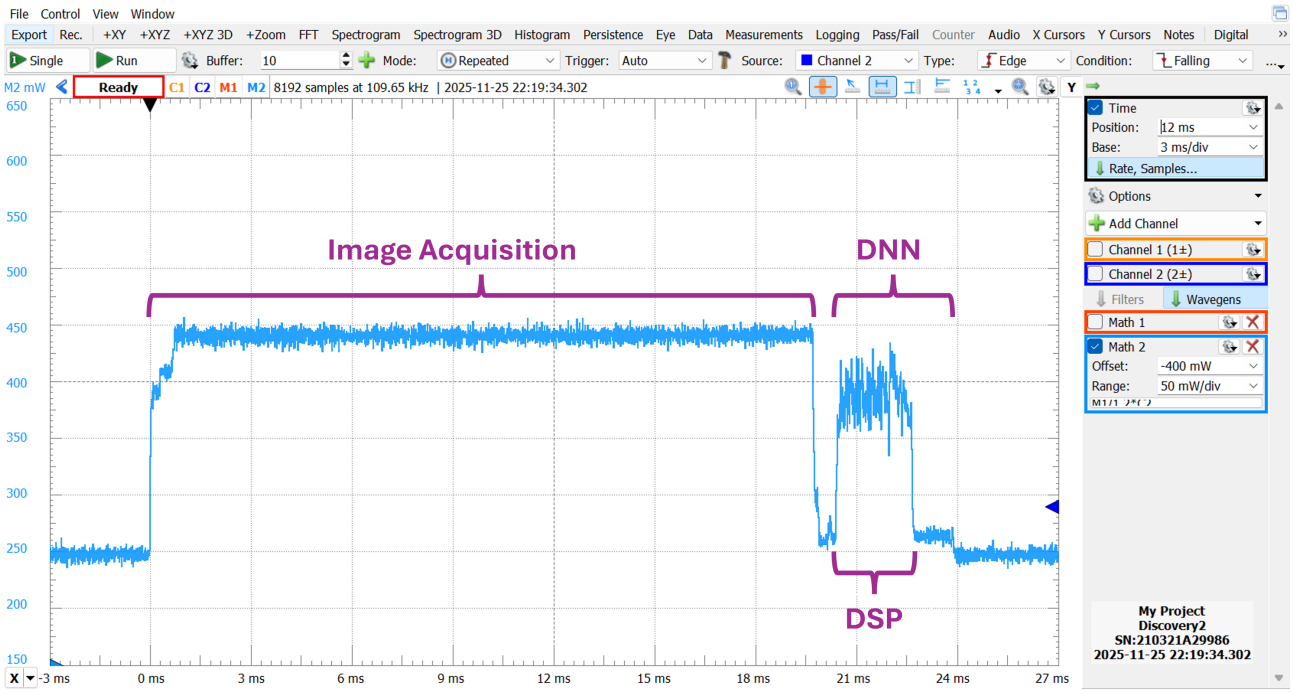


Figure 16: Measured power over a complete acquisition cycle, meaning first an image acquisition and then an inference.

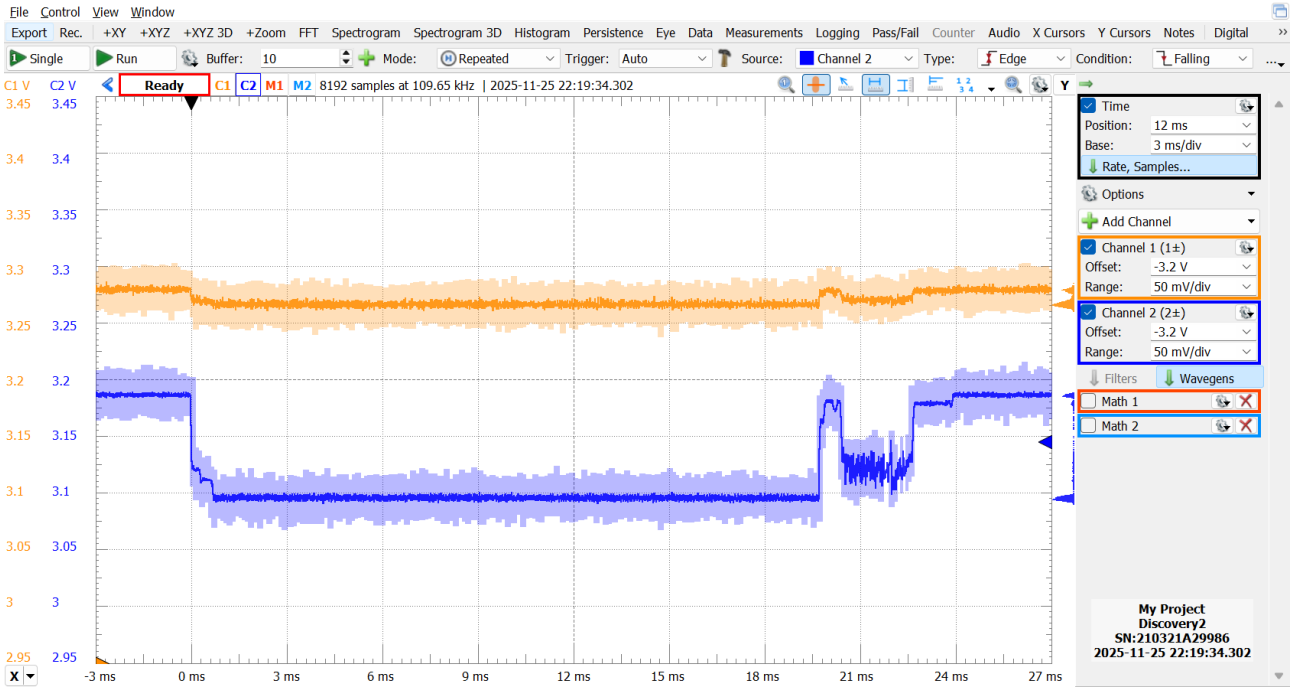


Figure 17: Measured voltage over a complete acquisition cycle, meaning first an image acquisition and then an inference.

parameters for training or the parameters of the post training quantization, aren't optimized at all. Rather, they provide a good starting point for your own solution.

Note: You can use this exercise and the provided Jupyter notebook as a template for your project. With this, you have a complete end to end flow, from the training of the model to running it on the IMX500 with the Raspberry Pi. Note that there are also other examples available in the Raspberry Pi repositories [a].

[a] https://github.com/raspberrypi/picamera2/tree/main/examples/imx500

Note: The provided workflow uses Google Colab to simplify the dependency management and installation procedure. Of course you are free to also use your own installation for the project, especially if there are no Google Colab GPU instances available for free. However, this is not recommended, as a proper setup often times requires a lot of time that otherwise could be spent on more interesting aspects of the project. Finally note, that during the creation of this exercise, a local installation was used first. Due to multiple confirmed dependency issues of Python packages on local installations, it was finally decided to move to Google Colab.

8.1 Further Ideas for Investigation

Some ideas that you can investigate with this exercise as starting point are:

- What is the influence of the chosen activation function on the performance on the IMX500? Do simple ReLU activation functions require less execution time than more complex continuous activation functions? How do they compare in terms of memory requirements?
- Which other transformations can be used for data augmentation? Can the parameters be further optimized? For hyperparameter selection Raytune[16] can be worth a look.
- Can we train the model as much as that it isn't under fitted anymore? Does a higher learning rate solve the shown issue?
- How does a model inference on the Raspberry Pi compare to a model inference on the IMX500? Is the IMX500 worth it over a conventional Raspberry Pi camera?
- How do quantization settings in the model compression toolkit influence our results? Which other techniques are available in the model compression toolkit? Which parameters can be optimized? For more about this, look at the example Google Colab notebooks linked in the readme of the model compression toolkit on GitHub[17].

$\mathcal{E}$ **Congratulations! You have reached the end of the exercise. If you are unsure of your results, discuss with an assistant.** $\mathcal{E}$

[16] https://docs.ray.io/en/latest/tune/index.html
[17] https://github.com/SonySemiconductorSolutions/mct-model-optimization/tree/main?tab=readme-ov-file